Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **How an agent works**
  - The agent loop, the model, and the harness
  - Tools, CLI, and MCP
- **How to direct its work**
  - Prompts, context, and persistent project instructions
  - Reusable procedures with skills
- **How to check the result**
  - Files, execution records, and reproducible outputs
  - One task throughout: turn a data file into a chart

The Large Language Model

A **large language model (LLM)** processes context and generates text, code, or structured tool requests.

- **OpenAI** develops the **GPT** model family; for example, GPT-6 Astra.
- **Anthropic** develops the **Claude** model family; for example, Claude Fable 5.

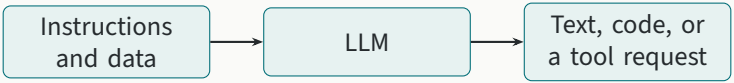

Figure 1: One model call produces an output from its input.

A running example

"Read `data.csv` and create a chart of daily temperature." The model can generate Python code or request a file-reading tool.

LLM + Harness = Agent

An **agent harness** runs an LLM in a loop with tools, context, and execution controls.

$$\underbrace{\text{LLM}}_{\text{selects actions}} + \underbrace{\text{harness}}_{\text{runs tools and manages context}} = \text{AI agent}$$

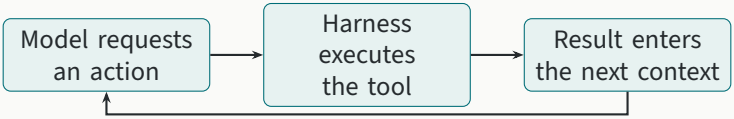

Figure 2: The agent loop repeats as work progresses.

Agent applications: OpenAI's Codex and Anthropic's Claude Code. Their harnesses manage tools, session state, and permissions around a supported model.

Chart task: read the CSV, write and run Python, inspect the chart, and revise it if needed.

A **sandbox** restricts execution; **approvals** authorize actions. Model choice and harness choice are separate decisions.

A Prompt Specifies the Task

A **prompt** supplies instructions for the current task. Specify the result, inputs, constraints, and checks.

A prompt for the chart task

> *Read* `data.csv`*, which contains* `date` *and* `temperature_c`*. Create* `plot_temperature.py` *to plot daily temperature in date order.*
>
> *Label the axes Date and Temperature (Celsius). Save the chart to* `results/temperature.png`*. Keep the input data unchanged.*
>
> *Run the script in the project environment and inspect the image. Check the dates, values, and units. Report the changed files, command used, and checks performed.*

The requested output and checks make completion observable. If the environment is unavailable, ask the agent to report the blocker.

Reading: OpenAI, Prompt engineering.

Context Is What the Model Can Use Now

Context is the input available to a model for its next response.

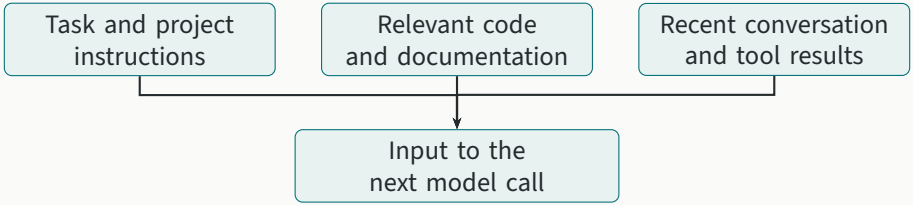

Figure 3: Context assembled for the next model call.

- Files become useful context when their relevant contents are loaded or retrieved.
- A **token** is a unit used to encode input and output. The **context window** limits how much a model call can accommodate.

Chart detail: reading `data.csv` reveals the column names; `temperature_c` records Celsius.

Keeping Context Useful

Context engineering means selecting and maintaining the information an agent needs as the task progresses.

- **Retrieve relevant material.** Read the CSV header and plotting script; retrieve errors after a failed run.
- **Keep durable decisions.** Save column meanings, units, and the run command in project files.
- **Summarize long sessions.** **Compaction** condenses earlier context; summaries can omit details, so retain important facts in files.

Handoff note

"The CSV uses Celsius. The script sorts dates and saves the chart. Next: inspect the axes and plotted values."

Memory stores information for retrieval across sessions; its scope depends on the application.

Reading: Anthropic, Effective context engineering for AI agents.

Tools, the Terminal, and CLI

A **tool** is a callable operation with inputs and a returned result: reading a file, editing code, or executing a command.

- A **CLI** (command-line interface) accepts text commands.
- The **terminal** hosts the session; the **shell** interprets commands.

Commands in the chart project

```
python plot_temperature.py
git diff
```

Run the script after creating it; use `git diff` to review edits in a Git repository.

The working directory and active Python environment determine which files and packages are used.

An agent CLI accepts your tasks in a terminal. Agents in an IDE or app can also use shell tools.

Reading: VS Code, Terminal Basics.

MCP Connects an Agent to Capabilities

Model Context Protocol (MCP) is a standard for communication between an AI application and servers that expose tools and context.

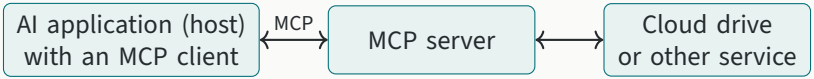

Figure 4: Connecting an application to a service through MCP.

- **Tools** perform operations; **resources** expose data; **prompts** provide reusable message templates.
- Servers run locally or remotely, with explicitly configured access.

Chart task: a cloud-drive MCP server could retrieve the CSV from a shared folder.

MCP standardizes the connection; the harness coordinates its use.

Reading: Model Context Protocol, Architecture overview.

Skills Package Reusable Procedures

A **skill** packages task instructions and optional scripts, references, or templates. In the Agent Skills format, its entry point is `SKILL.md`.

Example skill: plot a data file

```
plot-data/
  SKILL.md
  scripts/
  references/
```

Procedure in the skill:

1. Read column names and units.
2. Choose axes and create the plot.
3. Save and inspect the figure.
4. Report the command and checks.

Reading: Agent Skills, Overview and Specification.

When the chart task matches:

- The agent discovers the skill through its name and description.
- It loads the full instructions when relevant.
- It uses available tools to carry out the procedure.

The same procedure can guide other figures. Skills provide instructions; tools and permissions determine which actions can run.

Project Instructions Keep Conventions Consistent

Project instructions store guidance for work in a particular project. For example, Codex reads `AGENTS.md`; supported filenames and scope vary by tool.

Possible project guidance for charts

```
Use the existing project Python environment.
Keep input data unchanged.
Include units in axis labels.
Save generated images under results/.
Run the affected example after editing.
```

- The **task prompt** asks for this figure now.
- **Project instructions** describe conventions across tasks.
- A **skill** supplies a reusable procedure when the task calls for it.

Keep instructions short, current, and specific enough to check.

Reading: OpenAI, Custom instructions with AGENTS.md.

Checking an Agent's Work

An agent's completion message is a report. Check the underlying evidence.

1. **Inspect the edits.** Read the diff and check that edits match the plotting task.
2. **Reproduce the run.** Record the environment and command; confirm the output file exists.
3. **Check the result.** Compare plotted dates and values with the CSV; inspect labels and units.
4. **Keep a recoverable version.** Commit reviewed code with Git to compare or reverse later edits.

A concrete check

For three observations of 20, 22, and 21 Celsius, check that the chart preserves their date order and has its highest point at 22.

Treat instructions inside retrieved files as content to assess. **Prompt injection** uses such content to redirect an agent away from its task.

Practice: Turn a Data File into a Chart

Create `data.csv` in a working folder:

```
date,temperature_c
2026-10-01,20
2026-10-02,22
2026-10-03,21
```

1. Give the chart prompt to an agent with file and terminal access. In a chat-only workflow, attach the CSV and run the returned code yourself.
2. Identify the model, the agent application, the context it reads, and one tool call with its result.
3. Run `plot_temperature.py`; inspect `results/temperature.png` and review the script.

Follow-up: save the axis-unit convention as a project instruction. Package a repeated plotting procedure as a skill when supported.

File and shell tools are sufficient for this task. An MCP integration is useful when the CSV is stored in an external service.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - GPT-6 Astra Model
   - Codex as a platform: build on the open agent harness
   - Prompt engineering
   - Custom instructions with AGENTS.md
2. **Anthropic, Claude model and Claude Code documentation. [Documentation]**
   - Claude Fable 5
   - Claude Code: Overview
3. **Anthropic (2025), Effective context engineering for AI agents. [Technical article]**
   - Context engineering vs. prompt engineering

References and Further Reading

4. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
   - Terminal Basics
   - Secure AI-assisted development in VS Code
5. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview
6. **Agent Skills contributors, Agent Skills documentation. [Documentation]**
   - Agent Skills Overview
   - Specification